

Week-6 Spark Assignment

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer:

- **Driver:** The Driver is the main process of a Spark application. It creates the SparkSession, schedules tasks, and coordinates the execution of jobs.
 - **Cluster Manager:** The Cluster Manager allocates resources (CPU and memory) and manages the worker nodes in the cluster. Examples include Standalone, YARN, Mesos, and Kubernetes.
 - **Executor:** Executors run on worker nodes and execute the tasks assigned by the Driver. They process data and return the results to the Driver.
-

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer:

Spark uses **Lazy Evaluation**, which means transformations are not executed immediately. Instead, Spark builds a logical execution plan called a **DAG (Directed Acyclic Graph)**. The execution starts only when an action such as `show()`, `count()`, or `collect()` is called.

Advantages:

- Avoids unnecessary computations.
 - Optimizes multiple transformations into a single execution plan.
 - Reduces disk I/O.
 - Improves overall performance.
-

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("data/source.csv")
```

```
df.show()
```

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer:

CSV	Parquet
Row-based storage	Columnar storage
Larger file size	Compressed file format
Slower data reading	Faster data reading
No schema support	Preserves schema
Suitable for data exchange	Suitable for analytics

Why does it matter?

Parquet reads only the required columns instead of the entire dataset. This reduces disk I/O and improves query performance.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
result = (
    df.filter(df.category == "Electronics")
    .select("product_id", "price")
)

result.show()
```

Q6: Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

```
from pyspark.sql.functions import col
from pyspark.sql.types import DoubleType

df = (
    df.withColumnRenamed("old_name", "new_name")
    .withColumn("price", col("price").cast(DoubleType()))
)

df.show()
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer:

Spark maintains a **Lineage Graph (DAG)** that records all transformations applied to a DataFrame or RDD. If a worker node fails, Spark uses the lineage information to recompute only the lost partitions instead of reprocessing the entire dataset. This provides efficient fault tolerance.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

```
result = df_orders.filter(
    (df_orders.status == "Completed") &
    (df_orders.amount > 1000)
)

result.show()
```

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer:

Predicate Pushdown allows Spark to push filter conditions directly to the Parquet file reader. Instead of loading all the data into memory, Spark reads only the rows that satisfy the filter condition.

Benefits:

- Reduces disk I/O.
- Loads less data into memory.
- Improves query performance.

Q10: Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

df.show()
```

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Answer:

Transformations

Create a new DataFrame or RDD

Lazy evaluation

Actions

Trigger execution

Immediate execution

Transformation Examples:

- filter()
- select()

Action Examples:

- show()
- collect()

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
df = spark.read.parquet("path/to/input")

result = df.filter(df.user_id.isNotNull())

result.write \
    .option("header", True) \
    .csv("path/to/output")
```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer:

Client Mode	Cluster Mode
Driver runs on the client machine	Driver runs inside the cluster
Client must remain connected	Client can disconnect after job submission
Used for development and testing	Used for production environments
Less fault tolerant	More fault tolerant

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
result = df.filter(
    (df.region == "North") |
    (df.priority == "High")
)

result.show()
```

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer:

The `.show(5)` function displays only the first five rows of a dataset, making it fast and memory-efficient. On the other hand, `.collect()` retrieves the entire dataset to the Driver's memory. For a multi-terabyte dataset, this can consume excessive memory and may cause the Driver to crash with an **OutOfMemoryError**. Therefore, `.show(5)` is the safer option for exploring large datasets.